Project Title: Cloud-Based Payroll & Workforce Management System (Frontend Only)

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for the frontend interface of a cloud-based payroll and workforce management platform. This system will provide a responsive web interface designed to automate and visualize salary disbursement, attendance validation, task-based compensation, and leave management for organizations seeking transparent performance-linked payroll control.

1.2 Intended Audience

This document is intended for:

Project Developers.

UI/UX Designers.

Testers.

End Users (Admin, Managers, Employees).

1.3 Scope

The frontend system will provide a user-friendly dashboard and role-based management interface for workforce operations. The system will focus only on the frontend design and user interaction. Backend processing, active database implementation, and live cloud hosting are not included in this development scope.

1.4 Definitions

SRS:Software Requirement Specification.

UI: User Interface.

UX: User Experience.

HTML: HyperText Markup Language.

CSS: Cascading Style Sheets.

JS: JavaScript.

2. Overall Description

2.1 User Interface

The system will provide a responsive and attractive web interface.

UI Features will include:

Secure login dashboard.

Mobile-friendly responsive interface.

Sidebar Navigation.

Dashboard Cards.

Tables for Records.

Forms for Data Entry.

2.2 Constraints, Assumptions, and Dependencies

Constraints:

Frontend-only project.

No backend/database integration.

Static or dummy data used.

Dependencies:

JavaScript Runtime in Browser.

2.3 User Characteristics & Roles

User Type: Admin

Requires full system control UI.

Requires interfaces for employee onboarding/offboarding.

Requires payroll configuration views.

Requires reporting and audit logs dashboard.

User Type: Manager

Requires UI to assign tasks.

Requires UI to mark progress of assigned tasks as 10%, 20%, and so on.

Requires UI to validate attendance and remote check-ins.

Requires UI for salary approval authority.

User Type: User (Employee)

Requires UI to mark attendance from any location.

Requires UI to view salary, leaves, and task progress.

3. System Features and Requirements

3.1 Functional Requirements

3.1.1 Dashboard Module

Display role-based statistics (Admin, Manager, User).

Display pending task validations.

Display current leave ledger.

3.1.2 Task Assignment & Approval Module

Provide interfaces for Admin, Manager, and User to assign tasks.

Restrict task approval UI to Managers only.

Display task activity logs with simulated timestamps.

Limit task details to title and textual description with no subtasks.

3.1.3 Attendance Tracking Module

Provide UI for employees to mark attendance remotely from any location.

Simulate recording precise geographic coordinates.

Display attendance as pending until Manager approval.

3.1.4 Leave Management Module

Display visualization of 1 Paid Leave credited per month.

Display annual leave balance resets/cutoff on 1st April.

Provide UI for Manager approval required for leave usage.

Display full leave ledger.

3.1.5 Payroll & Salary Control Module

Display base visualization: 50% salary released by default to all employees.

Display visualization: Remaining 50% linked to approved task completion.

Simulate logic: 70% or more approved work completion results in 100% salary released.

Simulate logic: 50% to 69% completion results in 2 days salary retained temporarily.

Simulate logic: Below 50% completion results in 7 days salary retained until compliance or managerial decision.

3.2 Use Cases

Use Case 1: Manage Tasks

Actor: Manager.

Description: Mark progress of assigned tasks.

Precondition: Task is assigned.

Postcondition: Task progress updated to 10%, 20%, etc.

Use Case 2: Mark Attendance

Actor: User (Employee).

Description: Mark attendance from any location.

Precondition: User is logged in.

Postcondition: Attendance marked as pending until Manager approval.

3.3 External Interface Requirements

Responsive layout.

Clean navigation system.

Computer/Laptop and Mobile Device compatibility.

Modern Web Browser support.

3.4 Logical Data Structure (Dummy Data Representation)

Although the project is frontend only, the following logical data structure will be represented via mock JSON/JS objects:

Entity: Employee

Employee ID.

Name.

Role (Admin/Manager/User).

Entity: TaskTask ID.

Title and textual description.

Progress Percentage.

Entity: Attendance

Date.

Geo-location, timestamp, and device data.

Status (Pending/Approved).